État du backward — rien ne manque

Le backward bias est déjà fait via `cublasSgemv` avec `d_ones` — c'est la somme des gradients sur la dimension batch, ce qui est exactement `∂L/∂b = sum_batch(grad)`. Et `relu_back_kernel` est correct : `hidden` post-forward contient les valeurs post-ReLU, donc `x[idx] > 0` identifie correctement les neurones actifs. Tout est là.

FP16 + Tensor Cores — code complet

Le pattern standard : **master weights en FP32, compute en FP16**. Les Tensor Cores Ampere opèrent sur des tuiles 16×16×16 en FP16 avec accumulation FP32.

Struct modifiée

```
typedef struct {
    // Master weights FP32 (pour l'update SGD stable)
    float *weights1_fp32, *weights2_fp32, *weights3_fp32;
    float *bias1_fp32, *bias2_fp32, *bias3_fp32;

    // Shadow FP16 pour le compute
    __half *weights1, *weights2, *weights3;
    __half *bias1, *bias2, *bias3;

    // Gradients en FP32
    float *gradweights1, *gradweights2, *gradweights3;
    float *gradbias1, *gradbias2, *gradbias3;
} NeuralNetwork;
```

Kernel de conversion FP32 → FP16

```
__global__ void kernel_fp32_to_fp16(const float* src, __half* dst, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) dst[idx] = __float2half(src[idx]);
}

void sync_fp16_weights(NeuralNetwork* nn) {
    // Appelé après chaque weight update : recopie FP32 master → FP16 shadow
    int n1 = INPUT_SIZE * HIDDEN_SIZE;
    int n2 = HIDDEN_SIZE * HIDDEN_SIZE;
    int n3 = HIDDEN_SIZE * OUTPUT_SIZE;

    kernel_fp32_to_fp16<<<(n1+255)/256, 256>>>(nn->weights1_fp32, nn->weights1, n1);
    kernel_fp32_to_fp16<<<(n2+255)/256, 256>>>(nn->weights2_fp32, nn->weights2, n2);
```

```
    kernel_fp32_to_fp16<<<(n3+255)/256, 256>>>(nn->weights3_fp32, nn->weights3, n3);
    kernel_fp32_to_fp16<<<(HIDDEN_SIZE+255)/256, 256>>>(nn->bias1_fp32, nn->bias1,
HIDDEN_SIZE);
    kernel_fp32_to_fp16<<<(HIDDEN_SIZE+255)/256, 256>>>(nn->bias2_fp32, nn->bias2,
HIDDEN_SIZE);
    kernel_fp32_to_fp16<<<(OUTPUT_SIZE+255)/256, 256>>>(nn->bias3_fp32, nn->bias3,
OUTPUT_SIZE);
}
```

Remplacement des cublasSgemm

```
// alpha/beta sont float car CUBLAS_COMPUTE_32F
float alpha = 1.0f, beta = 0.0f;

// Remplace cublasSgemm pour W1 :
cublasGemmEx(handle,
    CUBLAS_OP_N, CUBLAS_OP_N,
    HIDDEN_SIZE, BATCH_SIZE, INPUT_SIZE,
    &alpha,
    nn->weights1,    CUDA_R_16F, HIDDEN_SIZE,
    batch_data_fp16, CUDA_R_16F, INPUT_SIZE,
    &beta,
    hidden,          CUDA_R_16F, HIDDEN_SIZE,
    CUBLAS_COMPUTE_32F_FAST_16F,  // Tensor Cores !
    CUBLAS_GEMM_DEFAULT_TENSOR_OP);
```

`CUBLAS_COMPUTE_32F_FAST_16F` : cuBLAS utilise les Tensor Cores avec accumulation FP32. C'est le sweet spot précision/vitesse sur Ampere.

Activations FP16 + kernels custom

```
// Buffers intermédiaires FP16
__half *dev_hidden_fp16, *dev_hidden2_fp16, *dev_output_fp16;
__half *batch_data_fp16;  // input converti en FP16

// kernel bias+relu fusionné FP16
__global__ void kernel_bias_relu_forward_fp16(__half* data, const __half* bias, int
hidden_size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int col = idx % hidden_size;
    int row = idx / hidden_size;
    if (row < BATCH_SIZE && col < hidden_size) {
        float val = __half2float(data[idx]) + __half2float(bias[col]);
        data[idx] = __float2half(val > 0.0f ? val : 0.0f);
    }
}
```

```
// relu backward doit comparer en FP16
__global__ void relu_back_kernel_fp16(const __half* x, float* dA, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        dA[idx] *= (__half2float(x[idx]) > 0.0f ? 1.0f : 0.0f);
    }
}
```

Loss scaling (indispensable en FP16 backward)

```
#define LOSS_SCALE 128.0f

// Dans la boucle d'entraînement, après crossentropy :
// 1. Multiplier la loss par LOSS_SCALE avant backward
// 2. Dans kernel_compute_output_gradients, diviser par LOSS_SCALE en plus de
BATCH_SIZE :

__global__ void kernel_compute_output_gradients_scaled(
    float* output, float* grad_output, int* batch_labels)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < BATCH_SIZE) {
        for (int i = 0; i < OUTPUT_SIZE; i++)
            grad_output[idx * OUTPUT_SIZE + i] = output[idx * OUTPUT_SIZE + i];
        grad_output[idx * OUTPUT_SIZE + batch_labels[idx]] -= 1.0f;
        for (int i = 0; i < OUTPUT_SIZE; i++)
            grad_output[idx * OUTPUT_SIZE + i] /= (BATCH_SIZE * LOSS_SCALE); // unscale
ici
    }
}
```

Boucle train modifiée

```
// Convertir le batch en FP16 une fois qu'il est sur GPU
kernel_fp32_to_fp16<<<(BATCH_SIZE*INPUT_SIZE+255)/256, 256>>>(
    dev_batch_fp32, batch_data_fp16, BATCH_SIZE * INPUT_SIZE);

forward(nn, handle, batch_data_fp16, dev_hidden_fp16, dev_hidden2_fp16,
dev_output_fp16);

// ... loss, backward avec gradients FP32 ...

// Weight update sur FP32 master
update_all_weights(nn, lr);

// Sync FP16 shadow
sync_fp16_weights(nn);
```

Init — allouer les deux copies

```
void init_NeuralNetwork(NeuralNetwork* nn) {
    // FP32 master
    CUDA_CHECK(cudaMalloc(&nn->weights1_fp32, INPUT_SIZE * HIDDEN_SIZE *
sizeof(float)));
    // ... etc pour tous les params ...

    // FP16 shadow
    CUDA_CHECK(cudaMalloc(&nn->weights1, INPUT_SIZE * HIDDEN_SIZE *
sizeof(__half)));
    // ... etc ...

    init_rand_weights(nn);      // initialise FP32
    sync_fp16_weights(nn);      // copie vers FP16
}
```

Backward bias — déjà correct, explication

La ligne `cublasSgemv(..., grad_output, OUTPUT_SIZE, d_ones, 1, ..., nn->gradbias3, 1)` calcule :

gradbias3 = grad_output (OUTPUT_SIZE × BATCH_SIZE) · ones (BATCH_SIZE)

C'est exactement `∂L/∂b3 = Σ_batch grad_output[b, :]` — somme des gradients sur tous les samples du batch. Identique pour gradbias1 et gradbias2. Rien ne manque.

Downloads pour le projet transformer

À faire maintenant (offline-friendly)

```
# TinyShakespeare — 1MB, parfait laptop
wget
https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
-O tiny_shakespeare.txt

# llm.c de Karpathy — référence absolue, lire pas copier
git clone https://github.com/karpathy/llm.c

# GPT-2 tokenizer vocab (pour tokenisation BPE offline)
```

```
wget https://openaipublic.blob.core.windows.net/gpt-2/models/124M/vocab.bpe
wget https://openaipublic.blob.core.windows.net/gpt-2/models/124M/encoder.json

# FineWeb-Edu sample 10B tokens — pour les 4090
# via HuggingFace datasets, script Python :
pip install datasets
python -c "
from datasets import load_dataset
ds = load_dataset('HuggingFaceFW/fineweb-edu', name='sample-10BT', split='train',
streaming=True)
# stream et sauvegarder en binary
"

# Poids GPT-2 small pour valider ton forward pass
python -c "import urllib.request;
urllib.request.urlretrieve('https://openaipublic.blob.core.windows.net/gpt-2/models/124M/mod
el.ckpt.index', 'gpt2_124M.index')"
# ou via HF :
python -c "from transformers import GPT2Model;
GPT2Model.from_pretrained('gpt2').save_pretrained('./gpt2_weights')"
```

Headers/libs pour coder offline

```
# Tout est déjà dans ton CUDA toolkit :
# /usr/local/cuda/include/cublas_v2.h
# /usr/local/cuda/include/cuda_fp16.h      ← FP16
# /usr/local/cuda/include/mma.h            ← wmma intrinsics Tensor Cores direct
# /usr/local/cuda/include/cublasLt.h       ← cuBLASLt pour fused ops

# Pour tokenisation en C pur (offline) :
git clone https://github.com/antirez/sds  # string lib légère
# ou juste utiliser le tokenizer de llm.c directement
```

PyTorch baseline à écrire maintenant

```
# benchmark_baseline.py — à avoir avant de coder quoi que ce soit
import torch, time

# GPT-2 small dims
B, T, C = 32, 256, 768      # batch, seq_len, d_model
d_ff = 4 * C                # 3072

# FFN seul (ta cible immédiate)
ffn = torch.nn.Sequential(
    torch.nn.Linear(C, d_ff),
    torch.nn.GELU(),
    torch.nn.Linear(d_ff, C)
```

```
).cuda().half()  # FP16

x = torch.randn(B, T, C, device='cuda', dtype=torch.float16)

# Warmup
for _ in range(10): y = ffn(x)
torch.cuda.synchronize()

# Benchmark
N = 100
t0 = time.perf_counter()
for _ in range(N): y = ffn(x)
torch.cuda.synchronize()
t1 = time.perf_counter()

tokens_per_sec = (B * T * N) / (t1 - t0)
print(f"PyTorch FP16 FFN : {tokens_per_sec/1e6:.2f}M tokens/sec")
print(f"Temps par forward : {(t1-t0)/N*1000:.3f}ms")
```

Lance ça sur ton laptop maintenant — ça te donne le chiffre à battre avant d'écrire une seule ligne de CUDA.

Plan du projet transformer.cu en une phrase

Dataset TinyShakespeare → tokenisation caractère (simple, pas BPE) → GPT architecture minimale → valider forward contre PyTorch couche par couche → profiler → optimiser les points chauds → scale sur 4× 4090 avec pipeline parallelism naïf. À chaque étape, un seul objectif : le kernel le plus lent dans ncu.

Pourquoi les poids GPT-2 — validation, pas training

Tu trains de zéro sur TinyShakespeare. Les poids GPT-2 servent uniquement à vérifier que ton forward pass est numériquement correct : tu charges les poids dans ton implem CUDA **et** dans PyTorch, tu passes le même input, tu compares les outputs couche par couche. Si tes valeurs matchent à 1e-3 près, ton implem est juste. Sans ça tu debugges dans le noir.

```
# validation workflow :
# 1. charger gpt2 weights dans pytorch
# 2. charger les mêmes weights dans ton buffer CUDA
# 3. même input → comparer hidden states à chaque layer
import numpy as np
ref_hidden = pytorch_forward(x)          # ground truth
your_hidden = np.fromfile("hidden.bin")  # dump depuis ton CUDA
assert np.allclose(ref_hidden, your_hidden, atol=1e-2)
```

L'architecture que tu vises

Un GPT decoder-only, le plus simple possible qui soit encore réel :

```
Input tokens (B, T)
   ↓
Token embedding  [vocab_size → d_model]     # lookup table
Position embedding [T → d_model]            # lookup table
   ↓
× N_layers :
   LayerNorm(x)
   + Self-Attention(x)      ← résiduel
   LayerNorm(x)
   + FFN(x)                 ← résiduel   ← TON MLP EST ICI
   ↓
LayerNorm final
Linear [d_model → vocab_size]
Softmax → proba sur vocab
```

Dimensions pour commencer petit sur laptop :

```
vocab_size = 65        # char-level TinyShakespeare (a-z, A-Z, ponctuation...)
d_model    = 128       # au lieu de 768 (GPT-2 small)
n_heads    = 4         # au lieu de 12
d_ff       = 4*128=512 # au lieu de 3072  ← ton HIDDEN_SIZE actuel !
n_layers   = 4         # au lieu de 12
seq_len    = 64        # au lieu de 1024
batch_size = 32
```

Ton MLP actuel (784→512→512→10) devient (128→512→128) avec GELU. Quasi pareil.

Dimensions pour les 4× 4090 :

```
vocab_size = 65 ou ~50k (BPE)
d_model    = 768
n_heads    = 12
d_ff       = 3072
n_layers   = 12
seq_len    = 512
batch_size = 64
```

Garder cuBLAS ou matmuls maison ?

Garde cuBLAS pour les GEMMs. Voilà pourquoi : dans un transformer, 90%+ du compute est dans les matmuls (QKV projections, attention scores, FFN). Si tu réécris les matmuls toi-même, tu passes 80% de ton temps à retrouver les perfs cuBLAS au lieu d'apprendre les trucs intéressants (FlashAttention, fused kernels, multi-GPU). Les kernels custom intéressants sont exactement là où cuBLAS ne peut pas aider : fusion bias+activation, LayerNorm, softmax causale masquée, FlashAttention.

Baseline PyTorch avec attention — syntaxe à compléter

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import time

# ---- hyperparams ----
B, T, C = 32, 64, 128   # batch, seq_len, d_model
n_heads  = 4
d_ff     = 4 * C
head_dim = C // n_heads  # 32

# ---- un seul bloc transformer ----
class TransformerBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1  = ...  # LayerNorm sur C
        self.ln2  = ...  # LayerNorm sur C

        # attention : Q, K, V projections + output proj
        self.qkv  = ...  # Linear(C, 3*C, bias=False)
        self.proj = ...  # Linear(C, C, bias=False)

        # FFN : deux linears + GELU
        self.ffn  = nn.Sequential(
            ...,   # Linear(C, d_ff)
            ...,   # GELU
            ...,   # Linear(d_ff, C)
        )

    def forward(self, x):
        # pre-norm + attention + résiduel
        x = x + self.attention(self.ln1(x))
        # pre-norm + FFN + résiduel
        x = x + self.ffn(self.ln2(x))
```

```
        return x

    def attention(self, x):
        B, T, C = x.shape

        # projeter Q, K, V d'un coup puis splitter
        qkv = self.qkv(x)                    # (B, T, 3*C)
        q, k, v = qkv.split(C, dim=-1)          # chacun (B, T, C)

        # reshaper pour multi-head : (B, n_heads, T, head_dim)
        q = q.view(B, T, n_heads, head_dim).transpose(1, 2)
        k = ...
        v = ...

        # scaled dot-product attention
        scale  = head_dim ** -0.5
        scores = torch.matmul(q, k.transpose(-2, -1)) * scale  # (B, n_heads, T, T)

        # causal mask : les tokens ne voient pas le futur
        mask   = torch.triu(torch.ones(T, T, device=x.device), diagonal=1).bool()
        scores = scores.masked_fill(mask, float('-inf'))

        attn   = F.softmax(scores, dim=-1)        # (B, n_heads, T, T)
        out    = torch.matmul(attn, v)           # (B, n_heads, T, head_dim)

        # recombiner les heads
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(out)

# ---- benchmark ----
block = TransformerBlock().cuda().half()
x     = torch.randn(B, T, C, device='cuda', dtype=torch.float16)

# warmup
for _ in range(20): _ = block(x)
torch.cuda.synchronize()

N  = 200
t0 = time.perf_counter()
for _ in range(N): y = block(x)
torch.cuda.synchronize()
elapsed = time.perf_counter() - t0

print(f"tokens/sec : {B*T*N/elapsed/1e6:.2f}M")
print(f"ms/block   : {elapsed/N*1000:.3f}ms")
```

Ce que tu dois compléter : les `...` — intentionnellement laissés.

Structure du projet transformer.cu

```
transformer/
├── data/
│   ├── tiny_shakespeare.txt
│   ├── prepare_data.py       # char→int, split train/val, écrire .bin
│   ├── X_train.bin
│   └── y_train.bin           # ici y[t] = x[t+1] (next token prediction)
│
├── reference/
│   ├── transformer_torch.py  # ton baseline PyTorch complet
│   └── dump_weights.py       # exporter poids PyTorch → .bin pour validation
│
├── src/
│   ├── transformer.cu        # fichier principal
│   ├── kernels/
│   │   ├── gelu.cuh          # __device__ float gelu(float x)
│   │   ├── layernorm.cuh     # kernel LayerNorm, un warp par token
│   │   ├── attention.cuh     # causal mask, softmax per head
│   │   └── softmax.cuh       # ton warp softmax déjà écrit
│   └── utils/
│       └── validate.cuh      # compare buffer CUDA vs numpy array chargé depuis .bin
│
└── Makefile
```

---

prepare_data.py — à écrire maintenant

```
import numpy as np

text = open('data/tiny_shakespeare.txt').read()
chars = sorted(set(text))
vocab_size = len(chars)         # 65 pour TinyShakespeare
print(f"vocab size: {vocab_size}")

stoi = {c: i for i, c in enumerate(chars)}
data = np.array([stoi[c] for c in text], dtype=np.int32)

# split 90/10
n     = int(0.9 * len(data))
train = data[:n]
val   = data[n:]

# pour l'entraînement : séquences de longueur (seq_len+1)
```

```
# input  = seq[0:seq_len]
# target = seq[1:seq_len+1]   ← next token prediction
SEQ_LEN = 64

# créer les batchs : à toi de coder le découpage et l'écriture en .bin
# hint : np.lib.stride_tricks.as_strided ou juste une boucle
# format final : X.bin shape (N_samples, SEQ_LEN) int32
#                y.bin shape (N_samples, SEQ_LEN) int32
```

Ordre d'implémentation dans transformer.cu

Étape 1 — GELU kernel

```
// Approximation de Hendrycks & Gimpel (utilisée dans GPT-2) :
// GELU(x) ≈ 0.5 * x * (1 + tanh(sqrt(2/π) * (x + 0.044715 * x³)))
// sqrt(2/π) ≈ 0.7978845608

__device__ float gelu(float x) { ... }

__global__ void kernel_gelu(float* data, int size) { ... }
```

Valide contre `torch.nn.functional.gelu(x)` avant de continuer.

Étape 2 — LayerNorm kernel

```
// Un bloc par token. Chaque bloc calcule mean et variance sur C valeurs,
// puis normalise : y = (x - mean) / sqrt(var + eps) * gamma + beta
// gamma et beta sont des paramètres appris (initialisés à 1 et 0)
// Utilise réduction warp comme ton softmax.

__global__ void kernel_layernorm(
    const float* x,       // (B*T, C)
    float* out,           // (B*T, C)
    const float* gamma,   // (C,)
    const float* beta,    // (C,)
    int C, float eps)
{
    // hint : si C > 32, tu as besoin de plusieurs warps par bloc
    // et de shared memory pour agréger les réductions inter-warps
    ...
}
```

Étape 3 — Embedding lookup

```
// Simple : chaque thread copie le vecteur d'embedding du token correspondant
// embedding_table : (vocab_size, d_model)
// tokens          : (B, T) int32
// out             : (B, T, d_model)

__global__ void kernel_embedding_lookup(
    const float* table, const int* tokens, float* out,
    int T, int d_model)
{ ... }
// + additionner positional embeddings (même structure)
```

Étape 4 — FFN block (ton MLP réutilisé)

```
// forward d'un FFN transformer :
// z = LayerNorm(x)
// h = GELU(z @ W1 + b1)    W1 : (d_model, d_ff)
// y = h @ W2 + b2          W2 : (d_ff, d_model)
// out = x + y              résiduel

void ffn_forward(
    cublasHandle_t handle,
    float* x,        // (B*T, d_model) — input et output résiduel
    float* z,        // buffer intermédiaire post-layernorm
    float* h,        // buffer intermédiaire post-gelu (B*T, d_ff)
    FFNWeights* w,
    int BT, int d_model, int d_ff)
{ ... }
```

Étape 5 — Attention

C'est la partie nouvelle. Structure du forward :

```
// Pour un bloc d'attention causal multi-head :
//
// 1. QKV projection : (B*T, C) @ W_qkv (C, 3C) → (B*T, 3C)    [cuBLAS]
// 2. Split + reshape → Q, K, V chacun (B, n_heads, T, head_dim)
// 3. Scores = Q @ K^T * scale                                  [cuBLAS ou kernel]
//    shape : (B, n_heads, T, T)
// 4. Causal mask kernel : mettre -inf sur upper triangle
// 5. Softmax sur dim=-1 (per row)                              [ton warp softmax adapté]
// 6. Out = softmax_scores @ V                                  [cuBLAS]
// 7. Reshape + output proj : (B*T, C) @ W_o (C, C)             [cuBLAS]
// 8. Résiduel : x = x + out

__global__ void kernel_causal_mask(float* scores, int T, int n_heads) {
    // scores : (B, n_heads, T, T) stocké en row-major
```

```
    // thread (i,j) : if j > i → scores[..., i, j] = -1e38f
    ...
}
```

Objectif mesurable à chaque étape

```
Étape 1 (GELU)     : output = torch_gelu(x) à atol=1e-4
Étape 2 (LayerNorm): output = torch_layernorm(x) à atol=1e-4
Étape 3 (FFN)      : throughput > 80% peak DRAM bandwidth (ncu)
Étape 4 (Attention): output = torch_attention(x) à atol=1e-3
Étape 5 (Full fwd) : tokens/sec > baseline PyTorch FP16 sur même hardware
```

Pour le dump de validation depuis PyTorch :

```
import torch, numpy as np

x = torch.randn(32, 64, 128).cuda().half()
block = TransformerBlock().cuda().half()

# forward et dump chaque activation intermédiaire
z1 = block.ln1(x)
z1.cpu().float().numpy().tofile('val_post_ln1.bin')

h_attn = block.attention(z1)
h_attn.cpu().float().numpy().tofile('val_post_attn.bin')
# ... etc
```

Depuis CUDA, tu charges le `.bin`, tu compares. Si ça matche à chaque couche, l'implémentation est correcte.

La progression — schéma simple

```
[MAINTENANT] MLP CUDA FP32 + cuBLAS
    accuracy ~97% MNIST, 1.5s/40 epochs
    kernels : sgemm + bias_relu + warp_softmax
    |
    | → valider accuracy vs PyTorch ✓ (déjà fait)
    |
```

```
[ÉTAPE A] MLP CUDA FP16 + Tensor Cores
   même accuracy, ~2x plus rapide sur les sgemm
   ce qui change : dtype des buffers + cublasGemmEx
   ce qui change pas : tous tes kernels custom, le backward, l'optimizer
   |
   | → valider que loss/accuracy sont identiques à FP32
   |
[ÉTAPE B] Remplacer MNIST par un vrai problème de séquence
   dataset : TinyShakespeare, char-level
   modèle : GPT minuscule (d=128, 4 heads, 4 layers)
   objectif accuracy : perplexité qui descend, texte généré lisible
   ce qui change : dataset, embedding lookup, loss (cross-entropy sur vocab)
   ce qui change pas : ton FFN (c'est exactement le même bloc)
   |
   | → avoir un truc qui génère du texte, même mauvais
   |
[ÉTAPE C] Ajouter self-attention
   ton FFN + LayerNorm + Attention = un bloc transformer complet
   objectif : même perplexité que PyTorch référence
   |
   | → valider numériquement couche par couche vs PyTorch
   |
[ÉTAPE D] Optimiser l'attention
   FlashAttention : ne jamais matérialiser la matrice T×T en DRAM
   objectif : battre torch.compile en tokens/sec
   |
[ÉTAPE E] Multi-GPU
   pipeline parallelism sur 4× RTX 4090
   layer i sur GPU i, cudaMemcpy D2D entre layers
```

Ce qui change pour FP16 + Tensor Cores

Ton code actuel a deux types d'objets : les **buffers** (hidden, output, batch_data) et les **appels cuBLAS**. Le reste ne bouge pas.

Les buffers — ce qui change

Actuellement tout est `float*`. Tu crées une version `__half*` en parallèle :

```
AVANT :
float* dev_hidden     (B × H floats)
float* nn->weights1   (I × H floats)

APRÈS :
float* dev_hidden_fp32    ← tu gardes ça pour le backward
```

```
__half* dev_hidden_fp16   ← nouveau, pour passer aux GEMMs
__half* nn->weights1_fp16 ← shadow FP16 des poids
float*  nn->weights1_fp32 ← master FP32, mis à jour par SGD
```

Pourquoi deux copies des poids ? SGD fait `w -= lr * grad`. Si `w` est FP16, l'update `lr * grad` underflow à 0 pour des petits gradients. Le master FP32 accumule correctement. Après chaque update, tu recopies FP32 → FP16 pour les GEMMs.

boucle d'entraînement :

```
forward  : utilise weights_fp16 + buffers fp16   → Tensor Cores
backward : calcule grads en FP32                 → rien ne change ici
update   : weights_fp32 -= lr * grads_fp32       → rien ne change
sync     : weights_fp32 → weights_fp16           → kernel de conversion
```

L'appel cuBLAS — ce qui change

```
// AVANT — un seul appel, tout FP32 :
cublasSgemm(handle,
    CUBLAS_OP_N, CUBLAS_OP_N,
    HIDDEN_SIZE, BATCH_SIZE, INPUT_SIZE,
    &alpha,
    nn->weights1,  HIDDEN_SIZE,   // float*
    batch_data,    INPUT_SIZE,    // float*
    &beta,
    hidden,        HIDDEN_SIZE);  // float*

// APRÈS — cublasGemmEx, dtype explicite :
cublasGemmEx(handle,
    CUBLAS_OP_N, CUBLAS_OP_N,
    HIDDEN_SIZE, BATCH_SIZE, INPUT_SIZE,
    &alpha,                          // float (pas __half !)
    nn->weights1_fp16, CUDA_R_16F, HIDDEN_SIZE,  // __half*
    batch_data_fp16,   CUDA_R_16F, INPUT_SIZE,   // __half*
    &beta,                           // float
    hidden_fp16,       CUDA_R_16F, HIDDEN_SIZE,  // __half* output
    CUBLAS_COMPUTE_32F_FAST_16F,         // accumulation FP32, Tensor Cores
    CUBLAS_GEMM_DEFAULT_TENSOR_OP);
```

`CUBLAS_COMPUTE_32F_FAST_16F` dit à cuBLAS : données en FP16, accumulation interne en FP32, utilise les Tensor Cores. C'est le seul flag qui change pour activer les Tensor Cores. cuBLAS choisit automatiquement le kernel Tensor Core optimal pour tes shapes.

Ce qui ne change absolument pas

Ton backward complet. Les `cublasSgemm` du backward calculent des gradients — ils restent en FP32 parce que les grads ont besoin de la précision. `relu_back_kernel`, `kernel_compute_output_gradients`, `cublasSgemv` pour les biais, `kernel_update_all_params` — tout ça reste identique.

Tes kernels custom `kernel_bias_relu_forward` et `softmax_kernel_warp` — ils deviennent des versions `__half` (juste changer le type et utiliser `__float2half`/`__half2float`), mais la logique est identique.

Par où commencer

Ne pars pas direct sur FP16 Tensor Cores. La raison : si l'accuracy baisse, tu ne sauras pas si c'est un bug de layout FP16 ou une vraie instabilité numérique. Le bon ordre :

1. Vérifie que ton FP32 actuel donne bien ~97% test accuracy
   (tu as implémenté evaluate() ? elle est vide dans ton code)

2. Passe en FP16 — vérifie que l'accuracy reste ~97%

3. Seulement là tu sais que FP16 est stable sur ce problème

La fonction `evaluate()` est vide dans ton code depuis le début. C'est ça le vrai trou. Tu mesures la loss d'entraînement mais tu ne sais pas si ton modèle généralise. Commence par ça :

```
void evaluate(NeuralNetwork* nn, cublasHandle_t handle,
              float* x_test_gpu, int* y_test_gpu) {
    // même structure que ta boucle train
    // mais : pas de backward, pas d'update
    // compte les prédictions correctes : argmax(output) == label
    // affiche accuracy sur les 10000 exemples de test
}
```

C'est 30 lignes. Tu ajoutes `argmax` kernel (trivial), tu appelles `evaluate()` après chaque epoch. Sans ça tu optimises à l'aveugle.